M2 평가 데이터셋 — 출처 · 가공 · 검증 과정

> 위치: `backend/m2/pj3_data/` 요지: **공신력 있는 오픈소스 벤치마크(OWASP BenchmarkPython, BugsInPy)를 원천으로 가져와**, 스펙(프롬프트) 기반으로 선별·변환하고, 자동 정합성 검증까지 거친 평가용 큐레이션 데이터셋을 생산했다. 자체 작성한 게 아니라 **검증된 git 벤치마크에서 출발**해 평가 신뢰도를 확보한 것이 핵심.

1. 원천 데이터 (공신력 근거)

원천	출처(git)	성격	규모	다운스트림
OWASP BenchmarkPython	`github.com/OWASP-Benchmark/BenchmarkPython`	OWASP 공식 — Python 취약점 탐지 도구 **정확도 측정 표준 벤치마크**	testcode 1,230건 + 정답 라벨 (`expectedresults-0.1.csv`) 1,230행	→ M2 OWASP selected **50**
BugsInPy	`github.com/soarsmu/BugsInPy` (`BugsInPy-master/`)	실제 OSS의 **재현 가능한 실버그** 벤치마크 (논문 기반)	17개 프로젝트 (ansible/black/fastapi/pandas/keras 등) + 재현 프레임워크	→ M3 BugsInPy selected **8**

- 원천은 `pj3_data/BenchmarkPython-main/`, `pj3_data/BugsInPy-master/`에 **수정 금지 원본**으로 보관(파이프라인 입력 전용).
- OWASP 원본은 Bandit SARIF / Bearer 실행 결과(`results/`)와 LICENSE까지 포함해 출처 추적성 유지.
- BugsInPy clone 캐시(`_repo_cache/`)는 `.gitignore` 처리.

2. 가공 파이프라인 (4단계)

```
[Stage 0] 원천 raw          [Stage 1] 정의(spec/prompt)        [Stage 2] 산출(curated)
[Stage 3] 검증
BenchmarkPython-main ─┐     m2_plus/ (OWASP 매핑 스펙)     ─┐
task_link_dataset_
 (OWASP 1230)         ├──▶  m2_dataset_split_files_v2/     ├──▶  datasets/ (큐레이션)
──▶  validation_prompt.md
BugsInPy-master      ─┘     bugsinpy_claude_code_request   ┘    (M2 62 / M3 28)
→ VALIDATION_REPORT
 (17 projects)
판정: PASS
```

단계	역할	입력 → 출력
0. 원천	외부 벤치마크 원본 보관	— → OWASP 1230, BugsInPy 17프로젝트
1. 정의	"어떻게 선별·가공할지" 스펙/프롬프트	원천+요구사항 → 케이스 생성 지시서
2. 산출	선별·변환된 평가 데이터셋	원천+스펙 → `datasets/` 큐레이션
3. 검증	정합성·완결성 자동 검사	산출물 → 검증 보고서(PASS)

스펙 묶음은 동일 패턴(`01_프롬프트` → `02_폴더구조` → `03_코드/매핑 스펙`):

- `m2_plus/` — OWASP 원본 → 50건 선별 + category→rule_id/CWE 매핑 규칙
- `m2_dataset_split_files_v2/` — M2 보안 취약점 custom 케이스 작성 규칙

3. M2 산출물 상세

(A) OWASP BenchmarkPython selected — 50건 (`datasets/m2_owasp_benchmark_python_selected/`)

선별 정책(`source_info.md`):

- 전체 1,230건을 통째로 복사하지 않고 **5개 카테고리 × (취약 5 + 정상 5) = 50건 선별**.
- **true/false 균형**: 각 카테고리 true 5 / false 5 → 오탐(FP)·미탐(FN)까지 측정 가능.
- 선별한 `testcode/BenchmarkTestXXXXX.py`와 정답 CSV 행을 1:1로만 가져옴.

category	CWE	rule_id	true	false
SQL Injection	CWE-89	SEC-001	5	5
XSS	CWE-79	SEC-002	5	5
Path Traversal	CWE-22	SEC-003	5	5
Command Injection	CWE-78	SEC-004	5	5
Weak Hash	CWE-328	SEC-005	5	5

산출 파일: `testcode/`(50), `expectedresults_selected.csv`, `expected_owasp_m2_issues.json`, `dataset_manifest.json`, `source_info.md`, `README.md`, `LICENSE`.

(B) M2 보안 custom — 단일 6 + mixed 6 (`datasets/m2_custom_security_cases/`)

OWASP가 못 덮는 사내 시나리오를 직접 작성. `vulnerable.py`/`fixed.py` 쌍 + `expected_issue.json`(Dev2 Output Contract 9필드).

유형	rule_id
sqli	SEC-001 / auth_missing
command_injection	SEC-004 / hardcoded_secret
insecure_password_hash	SEC-005 / path_traversal

- **mixed 6종**: 한 `vulnerable.py`에 3~5개 보안 이슈 혼합 → **다중 이슈 동시 탐지** 검증용.
- mixed에는 A커머스 도메인 전용 커스텀 규칙(`ecommerce_custom_issues`: ASEC-*/ASA-*)을 **비파괴(additive)** 로 부가, `concept_id`로 평가셋 SEC-*와 개념 매핑(M4 학습 연계).

4. 정답 라벨 & 스키마 정렬

- 정답은 `datasets/expected_outputs/expected_m2_issues.json` + OWASP 별도 `expected_owasp_m2_issues.json`.
- 케이스 이슈 객체는 m2 산출 계약(`scan_issues`)과 정렬: severity/rule_id/file/line/message/suggestion 공통 + M2 전용(taxonomy/concept_id/code_context 등 M4 학습용). 스키마 정의 `pj3_data/SCAN_ISSUES_COLUMNS.md`.
- rule_id→keyword 매핑(SEC-001=SQL Injection 등)으로 M4 커리큘럼까지 연결.

5. 검증 (Stage 3) — 자동 정합성 + pytest

`datasets/DATASET_VALIDATION_REPORT.md` (2026-06-04) — **판정 PASS**.

- **총 90 케이스** (M2 62 / M3 28). source 분포: OWASP 50 / BugsInPy 8 / Task-Link Custom 32.
  - (초기 78에서 M2 mixed 6 + M3 mixed 6 추가되어 90)
- 자동 스크립트 10항목 전부 PASS: 카운트 정합, case_id 중복 없음, `target_path`/`expected_output_ref` 90개 전부 실재, module/source 화이트리스트, manifest↔expected_issue rule_id 집합 일치.
- **OWASP true/false 균형 검증**: 5개 카테고리 모두 5/5 (전체 true 25 / false 25).
- N+1 확장 8케이스(positive 4=PERF-002, negative 4=[]) pytest 8 passed.
- 누락/경로불일치/파싱오류/라벨불균형 **전부 0건**.

6. 이 데이터로 무엇을 평가하나

- **정확도(Accuracy)**: 도구/LLM이 탐지한 이슈 vs 골든 정답 비교.
- **오탐·미탐(FP/FN)**: OWASP true/false 균형 덕에 정밀도/재현율까지 산출 가능.
- **다중 이슈 탐지**: mixed 케이스로 한 파일 내 복수 취약점 동시 검출 검증.
- LLM severity 재판정 평가(별도)는 `sub2_new/dev2/tests/eval/`의 골든셋과 연계.

부록: 핵심 경로

항목	경로
파이프라인 구조 설명	`backend/m2/pj3_data/DATA_PIPELINE_STRUCTURE.md`
검증 보고서	`backend/m2/pj3_data/datasets/DATASET_VALIDATION_REPORT.md`
OWASP 원천	`backend/m2/pj3_data/BenchmarkPython-main/`
BugsInPy 원천	`backend/m2/pj3_data/BugsInPy-master/`
OWASP selected 50	`backend/m2/pj3_data/datasets/m2_owasp_benchmark_python_selected/`
OWASP 출처/선별정책	`.../m2_owasp_benchmark_python_selected/source_info.md`
M2 보안 custom	`backend/m2/pj3_data/datasets/m2_custom_security_cases/`
통합 매니페스트	`backend/m2/pj3_data/datasets/dataset_manifest.json`
스키마 정의	`backend/m2/pj3_data/SCAN_ISSUES_COLUMNS.md`